

P1236R0: Alternative Wording for P0907R4 Signed Integers are Two's Complement

This paper presents alternative wording for P0907R3 Signed Integers are Two's Complement by Jean François Bastien, avoiding talking about unobservable bits as much as possible.

The wording presented here also resolved the following core issues:

- CWG 1857 Additional questions about bits
- CWG 1943 Unspecified meaning of "bit"

The change to [atomics.types.int] paragraph 8 resolves LWG issue 3047.

There are further cleanups in the area of integer types beyond requiring two's complement notation, not all of which Jean François Bastien may agree with. In particular, turning "Plain ints have the natural size suggested by the architecture of the execution environment" into a note in this paper appears contentious.

Roadmap to changes

First, the signed and unsigned integer types are introduced. Instead of discussing sign and value bits, the range of representable values is specified by introducing the term "range exponent". Then, "char" and narrow character types are introduced based on these. The concept of "underlying type" is extended to "char", consistent with the use for `wchar_t`.

The semantics of bitwise operators are reformulated based on the base-2 representation of the operand values.

Bit-fields now consistently use the term "width" (instead of "length").

Proposed wording

Change in 6.7.1 [basic.fundamental] paragraphs 1-7 and insert paragraph breaks as indicated:

Objects declared as characters (`char`) shall be large enough to store any member of the implementation's basic character set. If a character from this set is stored in a character object, the integral value of that character object is equal to the value of the single character literal form of that character. It is implementation-defined whether a `char` object can hold negative values. Characters can be explicitly declared unsigned or signed. Plain `char`, signed `char`, and unsigned `char` are three distinct types, collectively called narrow character types. A `char`, a signed `char`, and an unsigned `char` occupy the same amount of storage and have the same alignment requirements (6.6.5); that is, they have the same object representation. For narrow character types, all bits of the object representation participate in the value representation. [Note: A bit field of narrow character type whose length is larger than the number of bits in the object representation of that type has padding bits; see 6.7. -- end note] For unsigned narrow character types, each possible bit pattern of the value representation represents a distinct number. These requirements do not hold for other types. In any particular implementation, a plain `char` object can take on either the same values as a signed `char` or an unsigned `char`; which one is implementation-defined. For each value `i` of type unsigned `char` in the range 0 to 255 inclusive, there exists a value `j` of type `char`

such that the result of an integral conversion (7.3.8 [conv.integral]) from *i* to *char* is *j*, and the result of an integral conversion from *j* to *unsigned char* is *i*.

There are five *standard signed integer types*: "signed char", "short int", "int", "long int", and "long long int". In this list, each type provides at least as much storage as those preceding it in the list. There may also be implementation-defined *extended signed integer types*. The standard and extended signed integer types are collectively called *signed integer types*. **The range of representable values for a signed integer type is -2^{N-1} to $2^{N-1}-1$ (inclusive), where *N* is called the *range exponent* of the type.** [**Note:** Plain ints have the natural size suggested by the architecture of the execution environment [Footnote: *int* must also be large enough to contain any value in the range [INT_MIN, INT_MAX], as defined in the header <limits>.]; the other signed integer types are provided to meet special needs. **-- end note**]

For each of the standard signed integer types, there exists a corresponding (but different) *standard unsigned integer type*: "unsigned char", "unsigned short int", "unsigned int", "unsigned long int", and "unsigned long long int". **Likewise, for each of the extended signed integer types, there exists a corresponding extended unsigned integer type.** The standard and extended unsigned integer types are collectively called *unsigned integer types*. **The range of representable values for an unsigned integer type is 0 to 2^N-1 (inclusive), where *N* is called the *range exponent* of the type. Arithmetic for unsigned integer types is performed modulo 2^N .** [**Note:** Unsigned arithmetic does not overflow. Overflow for signed arithmetic yields undefined behavior (7.1 [expr.pre]). **-- end note**]

An unsigned integer type, each of which occupies the same amount of storage and has the same

- **object representation,**
- **value representation,**
- alignment requirements (6.6.5 [basic.align]), and
- **range exponent *N***

as the corresponding signed integer type. **For each value *x* of a signed integer type, there is a unique value *y* of the corresponding unsigned integer type such that *x* is congruent to *y* modulo 2^N , and vice versa; each such *x* and *y* have the same representation.** [**Footnote:** This is also known as two's complement representation.]. [**Example:** The value -1 of a signed type is congruent to the value 2^N-1 of the corresponding unsigned type; the representations are the same for these values.]

The minimum value required to be supported by the implementation for the range exponent of each signed integer type is specified in table X.

type	minimum range exponent <i>N</i>
signed char	8
short	16
int	16
long	32
long long	64

The value representation of a signed or unsigned integer type comprises *N* bits, where *N* is the respective range exponent. Each set of values for any padding bits (6.7 [basic.types]) in the object representation are alternative representations of the value specified by the value representation. [**Note:** Padding bits have unspecified value, but do not cause traps. See also ISO C 6.2.6.2. **-- end note**] [**Note:** The signed and unsigned integer types satisfy the constraints given in ISO C 5.2.4.2.1. **-- end note**] **Except as specified above, the range exponent of a signed or unsigned integer type is implementation-defined.** [Footnote: See 9.1.7.2 [del.type.simple] regarding the correspondence between types and the sequences of *type-specifiers* that designate them.]; that is, each signed integer type has the same object representation as its corresponding unsigned integer type. **Likewise, for each of the extended signed integer types there exists a corresponding extended unsigned integer type with the same amount of storage and alignment requirements. The standard and extended unsigned integer types are collectively called *unsigned integer types*.**

The range of non-negative values of a signed integer type is a subrange of the corresponding unsigned integer type, the representation of the same value in each of the two types is the same, and the value representation of each corresponding signed/unsigned type shall be the same.

Each value x of an unsigned integer type with range exponent N has a unique representation $x = x_0 2^0 + x_1 2^1 + \dots + x_{N-1} 2^{N-1}$, where each coefficient x_i is either 0 or 1; this is called the *base-2 representation* of x . The base-2 representation of a value of signed integer type is the base-2 representation of the congruent value of the corresponding unsigned integer type.

The standard signed integer types and standard unsigned integer types are collectively called the *standard integer types*, and the extended signed integer types and extended unsigned integer types are collectively called the *extended integer types*. The signed and unsigned integer types shall satisfy the constraints given in the C standard, subclause 5.2.4.2.1.

Unsigned integers shall obey the laws of arithmetic modulo 2^n where n is the number of bits in the value representation of that particular size of integer. [Footnote: This implies that unsigned arithmetic does not overflow because a result that cannot be represented by the resulting unsigned integer type is reduced modulo the number that is one greater than the largest value that can be represented by the resulting unsigned integer type.]

A fundamental type specified to have a signed or unsigned integer type as its *underlying type* has the same object representation, value representation, alignment requirements (6.6.5 [basic.align]), and range of representable values as the underlying type. Further, each value has the same representation in both types.

Type `char` is a distinct type that has an implementation-defined choice of "signed char" or "unsigned char" as its underlying type. The values of type `char` can represent distinct codes for all members of the implementation's basic character set. The three types `char`, `signed char`, and `unsigned char` are collectively called *narrow character types*. For narrow character types, each possible bit pattern of the object representation represents a distinct number. [Note: This requirement does not hold for other types. -- end note] [Note: A bit-field of narrow character type whose width is larger than the range exponent of that type has padding bits; see 6.7 [basic.types]. - end note]

Type `wchar_t` is a distinct type that has an implementation-defined signed or unsigned integer type as its underlying type. The values of type `wchar_t` whose values can represent distinct codes for all members of the largest extended character set specified among the supported locales (26.3.1 [locale]). Type `wchar_t` shall have the same size, signedness, and alignment requirements (6.6.5 [basic.align]) as one of the other integral types, called its underlying type. Types `char16_t` and `char32_t` denote distinct types with the same size, signedness, and alignment as whose underlying types are `uint_least16_t` and `uint_least32_t`, respectively, in `<stdint>`; called the underlying types.

Values of type `bool` are either true or false. [Footnote: Using a `bool` value in ways described by this document as "undefined", such as by examining the value of an uninitialized automatic object, might cause it to behave as if it is neither true nor false.] [Note: There are no signed, unsigned, short, or long `bool` types or values. -- end note] Values of type `bool` participate in integral promotions (7.3.6 [conv.prom]).

Types `bool`, `char`, `char16_t`, `char32_t`, `wchar_t`, and the signed and unsigned integer types are collectively called *integral types*. [Footnote: Therefore, enumerations (9.6 [decl.enum]) are not integral; however, enumerations can be promoted to integral types as specified in 7.3.6 [conv.prom].] A synonym for integral type is *integer type*. [Note: Enumerations (9.6 [decl.enum]) are not integral; however, unscoped enumerations can be promoted to integral types as specified in 7.3.6 [conv.prom]. -- end note] The representations of integral types shall define values by use of a pure binary numeration system. [Footnote: A positional representation for integers that uses the binary digits 0 and 1, in which the values represented by successive bits are additive, begin with 1, and are multiplied by successive

integral power of 2, except perhaps for the bit with the highest position. (Adapted from the American National Dictionary for Information Processing Systems.)] [Example: This document permits two's complement, ones' complement and signed magnitude representations for integral types. --- end example]

Change in [conv.integral] paragraphs 2-4:

If the destination type is unsigned, the resulting value is the least unsigned integer congruent to the source integer (modulo 2^n where n is the number of bits used to represent the unsigned type). [Note: In a two's complement representation, this conversion is conceptual and there is no change in the bit pattern (if there is no truncation). --- end note]

If the destination type is signed, the value is unchanged if it can be represented in the destination type; otherwise, the value is implementation-defined.

If the destination type is `bool`, see 7.3.14 [conv.bool]. If the source type is `bool`, the value `false` is converted to zero and the value `true` is converted to one.

Otherwise, the result is the unique value of the destination type that is congruent to the source integer modulo 2^N , where N is the range exponent of the destination type.

Change in 7.6.1.9 [expr.static.cast] paragraph 9:

A value of a scoped enumeration type (9.6) can be explicitly converted to an integral type. When that type is `cv bool`, the resulting value is `false` if the original value is zero and `true` for all other values. For the remaining integral types, the value is unchanged if the original value can be represented by the specified type. Otherwise, the resulting value is unspecified; **the result is the same as that of converting to the enumeration's underlying type and then to the destination type.** A value of a scoped enumeration type can also be explicitly converted to a floating-point type; the result is the same as that of converting from the original value to the floating-point type.

Change in 7.6.7 [expr.shift] paragraphs 1-4:

The shift operators `<<` and `>>` group left-to-right.

```
shift-expression:
    additive-expression
    shift-expression << additive-expression
    shift-expression >> additive-expression
```

The operands shall be of integral or unscoped enumeration type and integral promotions are performed. The type of the result is that of the promoted left operand. The behavior is undefined if the right operand is negative, or greater than or equal to the length in bits of the promoted left operand.

The value of $E1 \ll E2$ is **$E1 \times 2^{E2}$** [Note: $E1$ is left-shifted $E2$ bit positions; vacated bits are zero-filled. -- end note] If $E1$ has an unsigned type, the value of the result is $E1 \times 2^{E2}$, reduced modulo one more than the maximum value representable in the result type. Otherwise, if $E1$ has a signed type and non-negative value, and $E1 \times 2^{E2}$ is representable in the corresponding unsigned type of the result type, then that value, converted to the result type, is the resulting value; otherwise, the behavior is undefined.

The value of $E1 \gg E2$ is **$E1 / 2^{E2}$, rounded down.** [Note: $E1$ is right-shifted $E2$ bit positions. **Right-shift on signed integral types is an arithmetic right shift, which performs sign-extension.** - end note] If $E1$ has an unsigned type or if $E1$ has a signed type and a non-negative value, the value of the result is the integral part of the quotient of $E1/2^{E2}$. If $E1$ has a signed type and a negative value, the resulting value is implementation-defined.

The expression $E1$ is sequenced before the expression $E2$.

Change in 7.6.11 [expr.bit.and]:

```
and-expression:
    equality-expression
    and-expression & equality-expression
```

The operands shall be of integral or unscoped enumeration type. The usual arithmetic conversions (7.4 [expr.arith.conv]) are performed. **Given the coefficients x_i and y_i of the base-2 representation (6.7.1 [basic.fundamental]) of the converted operands x and y , the coefficient r_i of the base-2 representation of the result r is 1 if both x_i and y_i are 1, and 0 otherwise.** [Note: The result is the bitwise AND function of the operands. -- end note] ~~The operator applies only to integral or unscoped enumeration operands.~~

Change in 7.6.12 [expr.bit.xor]:

```
exclusive-or-expression:
    and-expression
    exclusive-or-expression ^ and-expression
```

The operands shall be of integral or unscoped enumeration type. The usual arithmetic conversions (7.4 [expr.arith.conv]) are performed. **Given the coefficients x_i and y_i of the base-2 representation (6.7.1 [basic.fundamental]) of the converted operands x and y , the coefficient r_i of the base-2 representation of the result r is 1 if either (but not both) of x_i and y_i are 1, and 0 otherwise.** [Note: ~~The~~ ; the result is the bitwise exclusive OR function of the operands. -- end note] ~~The operator applies only to integral or unscoped enumeration operands.~~

Change in 7.6.13 [expr.or]:

```
inclusive-or-expression:
    exclusive-or-expression
    inclusive-or-expression | exclusive-or-expression
```

The operands shall be of integral or unscoped enumeration type. The usual arithmetic conversions (7.4 [expr.arith.conv]) are performed. **Given the coefficients x_i and y_i of the base-2 representation (6.7.1 [basic.fundamental]) of the converted operands x and y , the coefficient r_i of the base-2 representation of the result r is 1 if at least one of x_i and y_i is 1, and 0 otherwise.** [Note: ~~The~~ ; the result is the bitwise inclusive OR function of its operands. -- end note] ~~The operator applies only to integral or unscoped enumeration operands.~~

Change in 9.6 [dcl.enum] paragraph 8:

For an enumeration whose underlying type is fixed, the values of the enumeration are the values of the underlying type. Otherwise, for an enumeration where $e_{\min}$ is the smallest enumerator and $e_{\max}$ is the largest, the values of the enumeration are the values in the range $b_{\min}$ to $b_{\max}$, defined as follows: ~~Let K be 1 for a two's complement representation and 0 for a ones' complement or sign-magnitude representation.~~ $b_{\max}$ is the smallest value greater than or equal to $\max(|e_{\min}| - K, e_{\max})$ and equal to 2^{M-1} , where M is a non-negative integer. $b_{\min}$ is zero if $e_{\min}$ is non-negative and $-(b_{\max} + K)$ otherwise. The **size width** of the smallest bit-field large enough to hold all the values of the enumeration type is $\max(M, 1)$ if $b_{\min}$ is zero and $M + 1$ otherwise. It is possible to define an enumeration that has values not defined by any of its enumerators. If the *enumerator-list* is empty, the values of the enumeration are as if the enumeration had a single enumerator with value 0. [Footnote: ...]

Change in 10.3.10 [class.bit] paragraph 1-4:

A *member-declarator* of the form

```
identifieropt attribute-specifier-seqopt : constant-expression brace-or-equal-initializeropt
```


specifies a bit-field; its length is set off from the bit-field name by a colon. The optional *attribute-specifier-seq* appertains to the entity being declared. **A bit-field shall not be a static member. A bit-field shall have integral or enumeration type; the** The bit-field attribute is not part of the type of the class member. The *constant-expression* shall be an integral constant expression with a value greater than or equal to zero **and is called the width of the bit-field. If the width of a bit-field is larger than the range exponent of the bit-field's type (or, in case of an enumeration type, of its underlying type),** The value of the integral constant expression may be larger than the number of bits in the object representation (6.7 [basic.types]) of the bit-field's type; in such cases the extra bits are padding bits (6.7 [basic.types]). Allocation of bit-fields within a class object is implementation-defined. Alignment of bit-fields is implementation-defined. Bit-fields are packed into some addressable allocation unit. [Note: Bit-fields straddle allocation units on some machines and not on others. Bit-fields are assigned right-to-left on some machines, left-to-right on others. -- end note]

A declaration for a bit-field that omits the *identifier* declares an *unnamed bit-field*. Unnamed bit-fields are not members and cannot be initialized. An unnamed bit-field shall not be declared with a cv-qualified type. [Note: An unnamed bit-field is useful for padding to conform to externally-imposed layouts. -- end note] As a special case, an unnamed bit-field with a width of zero specifies alignment of the next bit-field at an allocation unit boundary. Only when declaring an unnamed bit-field may the value of the constant-expression **width** be equal to zero.

~~A bit-field shall not be a static member. A bit-field shall have integral or enumeration type (6.7.1). A bool value can successfully be stored in a bit-field of any nonzero size.~~ The address-of operator & shall not be applied to a bit-field, so there are no pointers to bit-fields. A non-const reference shall not be bound to a bit-field (9.3.3). [Note: If the initializer for a reference of type const T& is an lvalue that refers to a bit-field, the reference is bound to a temporary initialized to hold the value of the bit-field; the reference is not bound to the bit-field directly. See 9.3.3. -- end note]

If a value of integral type (other than bool) is stored into a bit-field of width N and the value would be representable in a hypothetical signed or unsigned integer type with range exponent N and the same signedness as the bit-field's type, the original value and the value of the bit-field compare equal. If the value true or false is stored into a bit-field of type bool of any size (including a one bit bit-field), the original bool value and the value of the bit-field ~~shall~~ compare equal. If the value of an enumerator is stored into a bit-field of the same enumeration type and the ~~number of bits in the bit-field~~ **width** is large enough to hold all the values of that enumeration type (9.6 [dcl.enum]), the original enumerator value and the value of the bit-field ~~shall~~ compare equal. [Example: ...]

Change in 19.15.4.3 [meta.unary.prop] paragraph 9:

... The set of scalar types for which this condition holds is implementation-defined. [Note: If a type has padding bits, the condition does not hold; otherwise, the condition holds true for ~~unsigned~~ integral types. -- end note]

Change in 19.16.3 [ratio.ratio] paragraph 1:

If the template argument D is zero or the absolute values of either of the template arguments N and D is not representable by type intmax_t, the program is ill-formed. [Note: These rules ensure that infinite ratios are avoided and that for any negative input, there exists a representable value of its absolute value which is positive. ~~In a two's complement representation, this~~ **This** excludes the most negative value. -- end note]

Change in 29.7.2 [atomics.types.int] paragraph 7:

Remarks: For signed integer types, ~~arithmetic is defined to use two's complement representation.~~ **the result is as if the object value and parameters were converted to their corresponding unsigned types, the computation performed on those types, and the result converted back to the signed type.** [**Note:** There are no undefined results **arising from the computation.** -- end note]

Change in 29.7.2 [atomics.types.int] paragraph 8:

```
T operator op=(T operand) volatile noexcept;  
T operator op=(T operand) noexcept;
```

Effects: Equivalent to: return ~~fetch_key(operand) op operand;~~ static_cast<T>
(static_cast<make_unsigned_t<T>>(fetch_key(operand)) op
static_cast<make_unsigned_t<T>>(operand));